



University of Asia Pacific

Department of Computer Science and Engineering

CSE 438: Robotics Lab

Annexure (Code)

Submitted by:

Name: Dipu Banik
Student ID: 22201255

Name: Bivas Nandan Debnath
Student ID: 22201256

Name: Nazim Uddin
Student ID: 22201271

Name: Md. Sejon Hossain
Student ID: 22201272

Section: E2
Semester: 4th Year 1st Semester

Submitted to:

Tasnim Jahin Mowla
Lecturer
Department of Computer Science and Engineering

Project Code:

```
#include <WiFi.h>
#include <ESPmDNS.h>
#include <AsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <ESP32Servo.h>
#include <DHT.h>
#include <sstream>

// ===== WIFI =====

const char* ssid    = "Net_Error";
const char* password = "123456780";

String CAM_IP = "";
AsyncWebServer server(80);
AsyncWebSocket wsCtrl("/ctrl");

// ===== MOTOR =====

#define IN1 27
#define IN2 26
#define IN3 25
#define IN4 33
#define ENA 14
#define ENB 15

// ===== HARDWARE =====

#define PUMP    12
#define SERVO_PIN 13
#define RED_LED 18
```

```

#define GREEN_LED 19

#define BUZZER 17

// ===== SENSOR =====

#define FLAME_LEFT 34
#define FLAME_MIDDLE 32
#define FLAME_RIGHT 35
#define MQ2_PIN 16
#define DHTPIN 4
#define DHTTYPE DHT11

// ===== SETTINGS =====

#define FIRE_THRESHOLD 500
#define SMOKE_THRESHOLD 2000
#define MOTOR_SPEED 200
#define RELAY_ON HIGH
#define RELAY_OFF LOW

// ===== TIMING =====

#define MOVE_FORWARD_MS 200
#define TURN_LEFT_MS 200
#define TURN_RIGHT_MS 200
#define MOVE_BACKWARD_MS 100
#define STOP_DELAY_MS 50

// ===== SERVO ANGLES =====

#define SERVO_LEFT 45
#define SERVO_MIDDLE 90
#define SERVO_RIGHT 135

```

```

Servo sprayServo;
DHT dht(DHTPIN, DHTTYPE);

// ===== STATE =====

bool autoMode = true;
bool redLedState = false;
bool grnLedState = false;
bool buzzerState = false;

int servoAngle = 90;

float temperature = 0;
float humidity = 0;
int leftValue, middleValue, rightValue;

String modeStatus = "AUTO";
String robotStatus = "STOPPED";
String pumpStatus = "OFF";
String fireStatus = "SAFE";
String smokeStatus = "CLEAR";

// ===== mDNS =====

void resolveCamIP() {
    Serial.println("ESP32-CAM খোঁজা হচ্ছে...");
    IPAddress ip = MDNS.queryHost("firecam");
    if (ip != INADDR_NONE) {
        CAM_IP = ip.toString();
        Serial.println("CAM পাওয়া গেছে! IP: " + CAM_IP);
    } else {
        Serial.println("CAM পাওয়া যায়নি।");
    }
}

```

```

    CAM_IP = "";
}
}

// ===== MOTOR FUNCTIONS =====

void moveForward() {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
    analogWrite(ENA, MOTOR_SPEED);
    analogWrite(ENB, MOTOR_SPEED);
    robotStatus = "FORWARD";
}

void moveBackward() {
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
    analogWrite(ENA, MOTOR_SPEED);
    analogWrite(ENB, MOTOR_SPEED);
    robotStatus = "BACKWARD";
}

void turnLeft() {
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
    analogWrite(ENA, MOTOR_SPEED);
    analogWrite(ENB, MOTOR_SPEED);
    robotStatus = "LEFT";
}

void turnRight() {

```

```
digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);  
digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);  
analogWrite(ENA, MOTOR_SPEED);  
analogWrite(ENB, MOTOR_SPEED);  
robotStatus = "RIGHT";  
}
```

```
void slightLeft() {  
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);  
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);  
    analogWrite(ENA, MOTOR_SPEED * 0.6);  
    analogWrite(ENB, MOTOR_SPEED);  
    robotStatus = "SLIGHT_LEFT";  
}
```

```
void slightRight() {  
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);  
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);  
    analogWrite(ENA, MOTOR_SPEED);  
    analogWrite(ENB, MOTOR_SPEED * 0.6);  
    robotStatus = "SLIGHT_RIGHT";  
}
```

```
void stopMoving() {  
    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);  
    digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);  
    analogWrite(ENA, 0);  
    analogWrite(ENB, 0);  
    robotStatus = "STOPPED";  
}
```

```

// ===== PUMP =====

void pumpOn() { digitalWrite(PUMP, RELAY_ON); pumpStatus = "ON"; }
void pumpOff() { digitalWrite(PUMP, RELAY_OFF); pumpStatus = "OFF"; }


// ===== RESET ROBOT =====

void resetRobot() {
    moveBackward();
    delay(MOVE_BACKWARD_MS);
    stopMoving();
    pumpOff();
    digitalWrite(RED_LED, LOW);
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(BUZZER, LOW);
    servoAngle = 90;
    sprayServo.write(90);
}


// ===== FIRE FIGHT LOGIC =====

void firefightMode() {
    digitalWrite(RED_LED, HIGH);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(BUZZER, HIGH);

    if (middleValue < FIRE_THRESHOLD && rightValue < FIRE_THRESHOLD) {
        moveBackward(); delay(MOVE_BACKWARD_MS);
        stopMoving(); delay(STOP_DELAY_MS);
        slightRight(); delay(TURN_RIGHT_MS);
        stopMoving(); delay(STOP_DELAY_MS);
    }
}

```

```

else if (middleValue < FIRE_THRESHOLD && leftValue < FIRE_THRESHOLD) {
    moveBackward(); delay(MOVE_BACKWARD_MS);
    stopMoving(); delay(STOP_DELAY_MS);
    slightLeft(); delay(TURN_LEFT_MS);
    stopMoving(); delay(STOP_DELAY_MS);
}
else if (middleValue < FIRE_THRESHOLD) {
    moveForward(); delay(MOVE_FORWARD_MS);
    stopMoving(); delay(STOP_DELAY_MS);
}
else if (leftValue < FIRE_THRESHOLD) {
    moveBackward(); delay(MOVE_BACKWARD_MS);
    stopMoving(); delay(STOP_DELAY_MS);
    turnLeft(); delay(TURN_LEFT_MS);
    stopMoving();
}
else if (rightValue < FIRE_THRESHOLD) {
    moveBackward(); delay(MOVE_BACKWARD_MS);
    stopMoving(); delay(STOP_DELAY_MS);
    turnRight(); delay(TURN_RIGHT_MS);
    stopMoving();
}
else {
    stopMoving();
}

pumpOn();

for (int angle = SERVO_LEFT; angle <= SERVO_RIGHT; angle += 5) {
    servoAngle = angle; sprayServo.write(angle);
}

```

```

    for (int t = 0; t < 6; t++) { delay(10); wsCtrl.cleanupClients(); }
}

for (int angle = SERVO_RIGHT; angle >= SERVO_LEFT; angle -= 5) {
    servoAngle = angle; sprayServo.write(angle);
    for (int t = 0; t < 6; t++) { delay(10); wsCtrl.cleanupClients(); }
}

for (int angle = SERVO_LEFT; angle <= SERVO_MIDDLE; angle += 5) {
    servoAngle = angle; sprayServo.write(angle);
    for (int t = 0; t < 6; t++) { delay(10); wsCtrl.cleanupClients(); }
}

resetRobot();
}

// ===== STATUS JSON =====
String buildStatus() {
    String json = "{";
    json += "\"temp\":" + String(temperature, 1) + ",";
    json += "\"hum\":" + String(humidity, 1) + ",";
    json += "\"smoke\":" + smokeStatus + ",";
    json += "\"fire\":" + fireStatus + ",";
    json += "\"pump\":" + pumpStatus + ",";
    json += "\"robot\":" + robotStatus + ",";
    json += "\"mode\":" + modeStatus + ",";
    json += "\"servo\":" + String(servoAngle) + ",";
    json += "\"camip\":" + CAM_IP + ",";
    json += "}";
    return json;
}

```

```
// ===== WEBSOCKET HANDLER =====

void onCtrlEvent(AsyncWebSocket* s, AsyncWebSocketClient* client,
                 AwsEventType type, void* arg, uint8_t* data, size_t len)
{
    if (type == WS_EVT_CONNECT) {
        Serial.printf("WS connected #%u\n", client->id());
        client->text(buildStatus());

    } else if (type == WS_EVT_DISCONNECT) {
        Serial.printf("WS disconnected #%u\n", client->id());
        if (!autoMode) stopMoving();

    } else if (type == WS_EVT_DATA) {
        AwsFrameInfo* info = (AwsFrameInfo*)arg;
        if (info->final && info->index == 0 &&
            info->len == len && info->opcode == WS_TEXT)
        {
            String msg = String((char*)data, len);
            Serial.println("WS: " + msg);

            if (msg == "forward") { if (!autoMode) moveForward(); }
            else if (msg == "backward") { if (!autoMode) moveBackward(); }
            else if (msg == "left") { if (!autoMode) turnLeft(); }
            else if (msg == "right") { if (!autoMode) turnRight(); }
            else if (msg == "stop") { stopMoving(); }
            else if (msg == "auto") { autoMode = true; modeStatus = "AUTO"; }
            else if (msg == "manual") { autoMode = false; modeStatus = "MANUAL"; stopMoving(); }
            else if (msg == "pumpon") { if (!autoMode) pumpOn(); }
            else if (msg == "pumpoff") { if (!autoMode) pumpOff(); }
        }
    }
}
```

```

    else if (msg == "ledrednon") { if (!autoMode) { redLedState=true; digitalWrite(RED_LED, HIGH); }
}
    else if (msg == "ledredoff") { if (!autoMode) { redLedState=false; digitalWrite(RED_LED, LOW); }
}
    else if (msg == "ledgrnon") { if (!autoMode) { grnLedState=true; digitalWrite(GREEN_LED,HIGH);
} }
    else if (msg == "ledgrnoff") { if (!autoMode) { grnLedState=false;
digitalWrite(GREEN_LED,LOW); } }

    else if (msg == "buzzeron") { if (!autoMode) { buzzerState=true; digitalWrite(BUZZER,HIGH); } }
    else if (msg == "buzzeroff") { if (!autoMode) { buzzerState=false; digitalWrite(BUZZER,LOW); } }
    else if (msg.startsWith("servo,")) {
        if (!autoMode) {
            int angle = constrain(msg.substring(6).toInt(), 0, 180);
            int mapped = map(angle, 0, 180, 180, 0);
            servoAngle = angle;
            sprayServo.write(mapped);
        }
    }
    else if (msg == "refreshcam") {
        resolveCamIP();
        client->text(buildStatus());
    }
}
}
}
}

```

```
// ===== HTML PAGE =====
```

```

const char* htmlPage PROGMEM = R"HTMLPAGE(
<!DOCTYPE html>
<html>
<head>

```

```

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1">

<title>Fire Fighter Robot</title>

<style>

  * { box-sizing: border-box; margin: 0; padding: 0; }

  body { font-family: Arial, sans-serif; background: #1a1a2e; color: #eee; min-height: 100vh; padding: 12px; }

  h1 { text-align: center; color: #ff6b35; font-size: 22px; margin-bottom: 14px; letter-spacing: 1px; }

  .main-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; max-width: 900px; margin: auto; }

  @media (max-width: 600px) { .main-grid { grid-template-columns: 1fr; } }

  .card { background: #16213e; border-radius: 12px; padding: 14px; border: 1px solid #0f3460; }

  .card h3 { color: #e94560; font-size: 13px; text-transform: uppercase; letter-spacing: 1px; margin-bottom: 10px; border-bottom: 1px solid #0f3460; padding-bottom: 6px; }

  .full-width { grid-column: 1 / -1; }

  .cam-warning { font-size: 13px; color: #fbbf24; text-align: center; padding: 20px; background: #1a1200; border-radius: 8px; }

  .cam-status { font-size: 11px; color: #888; margin-top: 6px; text-align: center; }

  .ws-status { font-size: 11px; color: #555; margin-bottom: 10px; display: flex; align-items: center; gap: 6px; }

  .dot { width: 8px; height: 8px; border-radius: 50%; background: #f87171; flex-shrink: 0; }

  .dot.on { background: #4ade80; }

  .pantilt-toggle { width: 100%; padding: 8px 12px; margin-top: 10px; background: #0f3460; color: #60a5fa; border: 1px solid #1e3a5f; border-radius: 8px; font-size: 12px; cursor: pointer; text-align: left; }

  .pantilt-toggle:active { background: #1e3a5f; }

  .pantilt-body { display: none; margin-top: 8px; }

  .pantilt-body.open { display: block; }

  .slider-wrap { padding: 10px; background: #0f3460; border-radius: 8px; margin-bottom: 8px; }

  .slider-label { font-size: 11px; color: #888; text-transform: uppercase; margin-bottom: 6px; display: flex; justify-content: space-between; align-items: center; }

  .slider-label span { color: #fbbf24; font-size: 14px; font-weight: bold; }

  input[type=range] { width: 100%; accent-color: #e94560; cursor: pointer; }

```

```

.slider-ends { display: flex; justify-content: space-between; font-size: 10px; color: #666; margin-top: 2px; }

.reset-btn { width: 100%; padding: 8px; background: #1e3a5f; color: #60a5fa; border: 1px solid #2563eb; border-radius: 8px; font-size: 12px; cursor: pointer; margin-top: 4px; }

.reset-btn:active { background: #2563eb; color: #fff; }

.status-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 8px; }

.status-item { background: #0f3460; border-radius: 8px; padding: 10px; text-align: center; }

.status-label { font-size: 10px; color: #888; text-transform: uppercase; margin-bottom: 4px; }

.status-value { font-size: 16px; font-weight: bold; }

.val-safe { color: #4ade80; } .val-danger { color: #f87171; }

.val-warn { color: #fbbf24; } .val-info { color: #60a5fa; } .val-neutral { color: #e2e8f0; }

.dpad { display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px; max-width: 200px; margin: auto; }

.dpad-btn { background: #0f3460; border: 1px solid #e94560; color: #fff; border-radius: 8px; height: 50px; font-size: 22px; cursor: pointer; transition: background 0.1s; -webkit-user-select: none; user-select: none; touch-action: none; }

.dpad-btn:active { background: #e94560; }

.dpad-center { background: #1a1a2e; border-color: #333; }

.btn-row { display: flex; gap: 8px; flex-wrap: wrap; justify-content: center; margin-top: 10px; }

.action-btn { padding: 10px 16px; border-radius: 8px; border: none; font-size: 12px; font-weight: bold; cursor: pointer; min-width: 80px; }

.btn-off { background: #374151; color: #fff; } .btn-auto { background: #10b981; color: #000; }

.btn-manual { background: #f59e0b; color: #000; } .btn-stop { background: #ef4444; color: #fff; }

.btn-pump-on { background: #3b82f6; color: #fff; } .btn-pump-off { background: #374151; color: #fff; }

.mode-badge { display: inline-block; padding: 4px 12px; border-radius: 20px; font-size: 12px; font-weight: bold; margin-bottom: 10px; }

.badge-auto { background: #10b981; color: #000; } .badge-manual { background: #f59e0b; color: #000; }

.btn-active-red { background: #ef4444 !important; color: #fff !important; }

.btn-active-green { background: #4ade80 !important; color: #000 !important; }

.btn-active-buzz { background: #fbbf24 !important; color: #000 !important; }

.servo-wrap { margin-top: 12px; padding: 10px; background: #0f3460; border-radius: 8px; }

```

```

.servo-label { font-size: 11px; color: #888; text-transform: uppercase; margin-bottom: 6px; display: flex;
justify-content: space-between; align-items: center; }

.servo-label span { color: #fbbf24; font-size: 14px; font-weight: bold; }

.servo-ends { display: flex; justify-content: space-between; font-size: 10px; color: #666; margin-top:
2px; }

.servo-auto-info { margin-top: 12px; padding: 10px; background: #0f3460; border-radius: 8px; text-
align: center; }

.servo-auto-label { font-size: 11px; color: #888; text-transform: uppercase; margin-bottom: 4px; }

.servo-angle-display { font-size: 18px; font-weight: bold; color: #fbbf24; }

</style>
</head>
<body>
<h1>🔥 Fire Fighter Robot</h1>
<div class="main-grid">

<!-- CAMERA -->
<div class="card full-width">
<h3>📹 Live Camera</h3>
<div class="ws-status">
<span class="dot" id="dot"></span>
<span id="wsStatusText">Connecting...</span>
</div>
<div id="camContainer">
<div class="cam-warning">⚠️ ESP32-CAM পাওয়া যায়নি</div>
</div>
<button class="pantilt-toggle" id="ptToggleBtn" onclick="togglePanTilt()">⚙️ Pan / Tilt / Flash
🔦</button>
<div class="pantilt-body" id="panTiltBody">
<div class="slider-wrap">
<div class="slider-label">📏 Pan 📏<span id="panVal">90°</span></div>
<input type="range" min="0" max="180" value="90" id="panSlider"
oninput="onPanSlide(this.value)">

```

```
<div class="slider-ends"><span>বামে (0°)</span><span>মাঝে (90°)</span><span>ডানে (180°)</span></div>
```

```
</div>
```

```
<div class="slider-wrap">
```

```
<div class="slider-label">Tilt<span id="tiltVal">90°</span></div>
```

```
<input type="range" min="0" max="180" value="90" id="tiltSlider"
oninput="onTiltSlide(this.value)">
```

```
<div class="slider-ends"><span>উপরে (0°)</span><span>মাঝে (90°)</span><span>নিচে (180°)</span></div>
```

```
</div>
```

```
<div class="slider-wrap">
```

```
<div class="slider-label">Flash<span id="flashVal">OFF</span></div>
```

```
<input type="range" min="0" max="255" value="0" id="flashSlider"
oninput="onFlashSlide(this.value)">
```

```
</div>
```

```
<button class="reset-btn" onclick="resetPanTilt()">Pan/Tilt Center</button>
```

```
</div>
```

```
</div>
```

```
<!-- SENSOR STATUS -->
```

```
<div class="card">
```

```
<h3>Sensor Status</h3>
```

```
<div class="status-grid">
```

```
<div class="status-item"><div class="status-label">Temperature</div><div class="status-value val-
info" id="temp">--°C</div></div>
```

```
<div class="status-item"><div class="status-label">Humidity</div><div class="status-value val-
info" id="hum">--%</div></div>
```

```
<div class="status-item"><div class="status-label">Smoke</div><div class="status-value"
id="smoke">--</div></div>
```

```
<div class="status-item"><div class="status-label">Fire</div><div class="status-value" id="fire">--
</div></div>
```

```
<div class="status-item"><div class="status-label">Pump</div><div class="status-value"
id="pump">--</div></div>
```

```

    <div class="status-item"><div class="status-label">Robot</div><div class="status-value"
id="robot">--</div></div>

</div>

</div>

<!-- ROBOT CONTROL -->

<div class="card">

    <h3>&#127918; Robot Control</h3>

    <div style="text-align:center; margin-bottom:10px;">

        <span id="modeBadge" class="mode-badg badge-auto">AUTO MODE</span>

    </div>

    <div class="dpad">

        <div></div>

        <button class="dpad-btn"

            ontouchstart="pressStart('forward')" ontouchend="pressEnd()" ontouchcancel="pressEnd()"

            onmousedown="pressStart('forward')" onmouseup="pressEnd()"

onmouseleave="pressEnd()">&#8679;</button>

        <div></div>

        <button class="dpad-btn"

            ontouchstart="pressStart('left')" ontouchend="pressEnd()" ontouchcancel="pressEnd()"

            onmousedown="pressStart('left')" onmouseup="pressEnd()"

onmouseleave="pressEnd()">&#8678;</button>

        <button class="dpad-btn dpad-center" ontouchstart="send('stop')"

onmousedown="send('stop')">&#9632;</button>

        <button class="dpad-btn"

            ontouchstart="pressStart('right')" ontouchend="pressEnd()" ontouchcancel="pressEnd()"

            onmousedown="pressStart('right')" onmouseup="pressEnd()"

onmouseleave="pressEnd()">&#8680;</button>

        <div></div>

        <button class="dpad-btn"

            ontouchstart="pressStart('backward')" ontouchend="pressEnd()" ontouchcancel="pressEnd()"

```

```

        onmousedown="pressStart('backward')"
```

```

onmouseleave="pressEnd()">&#8681;</button>
        onmouseup="pressEnd()"

    <div></div>

</div>

<div class="btn-row">

    <button class="action-btn btn-pump-on"  onclick="send('pumpon')">&#128167; Pump ON</button>

    <button class="action-btn btn-pump-off"  onclick="send('pumpoff')">Pump OFF</button>

</div>

<div class="btn-row">

    <button class="action-btn btn-auto"  onclick="setMode('auto')">&#129302; AUTO</button>

    <button class="action-btn btn-manual"  onclick="setMode('manual')">&#9881; MANUAL</button>

    <button class="action-btn btn-stop"  onclick="send('stop')">&#9940; STOP</button>

</div>

</div>

<!-- MANUAL HARDWARE CONTROL -->

<div class="card full-width">

    <h3>&#9881; Manual Hardware Control</h3>

    <div class="btn-row">

        <div style="text-align:center;">

            <div style="font-size:11px; color:#888; margin-bottom:5px;">RED LED</div>

            <button id="btnRedOn"  class="action-btn btn-off"  onclick="toggleLed('red','on')">&#128308;
ON</button>

            <button id="btnRedOff"  class="action-btn btn-off"  onclick="toggleLed('red','off')" style="margin-
left:4px;">OFF</button>

        </div>

        <div style="text-align:center;">

            <div style="font-size:11px; color:#888; margin-bottom:5px;">GREEN LED</div>

            <button id="btnGrnOn"  class="action-btn btn-off"  onclick="toggleLed('grn','on')">&#128994;
ON</button>

            <button id="btnGrnOff"  class="action-btn btn-off"  onclick="toggleLed('grn','off')" style="margin-
left:4px;">OFF</button>

        </div>

    </div>

</div>

```

```

</div>

<div style="text-align:center;">
  <div style="font-size:11px; color:#888; margin-bottom:5px;">BUZZER</div>
  <button id="btnBuzzOn" class="action-btn btn-off" onclick="toggleBuzz('on')">&#128276;
ON</button>
  <button id="btnBuzzOff" class="action-btn btn-off" onclick="toggleBuzz('off')" style="margin-
left:4px;">OFF</button>
</div>
</div>

<div id="servoManual" class="servo-wrap">
  <div class="servo-label">Nozzle Direction (Servo)<span id="servoVal">90°</span></div>
  <input type="range" id="servoSlider" min="0" max="180" value="90"
  oninput="onServoSlide(this.value)">
  <div class="servo-ends"><span>&#8678; বামে (0°)</span><span>মাঝে (90°)</span><span>ডানে
(180°) &#8680;</span></div>
</div>

<div id="servoAuto" class="servo-auto-info" style="display:none;">
  <div class="servo-auto-label">Nozzle — Auto Sweep</div>
  <div class="servo-angle-display" id="servoAutoVal">90°</div>
  <div style="font-size:11px; color:#666; margin-top:4px;">Fire detect হলে servo 45°→135°→90°
sweep করে</div>
</div>
</div>

<script>
var ws = null;
var wsOk = false;

```

```

var wsCam = null;
var camOk = false;
var lastCamIp = "";
var currentMode = 'auto';
var holdInterval = null;
var lastServoSend = 0;
var lastPanSend = 0, lastTiltSend = 0, lastFlashSend = 0;

// ===== Robot WebSocket =====

function initWS() {
    ws = new WebSocket('ws://' + window.location.hostname + '/ctrl');
    ws.onopen = function() { wsOk = true; setDot(true, 'Connected'); };
    ws.onclose = function() { wsOk = false; setDot(false, 'Disconnected — reconnecting...');
    setTimeout(initWS, 2000); };
    ws.onerror = function() { wsOk = false; setDot(false, 'Error'); };
    ws.onmessage = function(e) { try { updateUI(JSON.parse(e.data)); } catch(err) {} };
}

function send(msg) {
    if (wsOk && ws && ws.readyState === WebSocket.OPEN) ws.send(msg);
}

function setDot(ok, txt) {
    document.getElementById('dot').className = 'dot' + (ok ? ' on' : '');
    document.getElementById('wsStatusText').textContent = txt;
}

// ===== CAM WebSocket (Pan/Tilt/Flash) =====

function initCamWS(ip) {
    if (wsCam) { wsCam.close(); wsCam = null; }

```

```

wsCam = new WebSocket('ws://' + ip + '/CarInput');
wsCam.onopen = function() { camOk = true; };
wsCam.onclose = function() { camOk = false; };
wsCam.onerror = function() { camOk = false; };
}

function sendCam(key, val) {
    if (camOk && wsCam && wsCam.readyState === WebSocket.OPEN)
        wsCam.send(key + ',' + val);
}

// ===== Pan/Tilt toggle =====
function togglePanTilt() {
    var body = document.getElementById('panTiltBody');
    var btn = document.getElementById('ptToggleBtn');
    body.classList.toggle('open');
    btn.textContent = body.classList.contains('open')
        ? '\u2699 Pan / Tilt / Flash \u25b2'
        : '\u2699 Pan / Tilt / Flash \u25bc';
}

// ===== Pan/Tilt/Flash sliders =====
function onPanSlide(val) {
    document.getElementById('panVal').textContent = val + '\u00b0';
    var now = Date.now();
    if (now - lastPanSend >= 30) { lastPanSend = now; sendCam('Pan', val); }
}

function onTiltSlide(val) {
    document.getElementById('tiltVal').textContent = val + '\u00b0';
    var now = Date.now();

```

```

    if (now - lastTiltSend >= 30) { lastTiltSend = now; sendCam('Tilt', val); }
}

function onFlashSlide(val) {
    document.getElementById('flashVal').textContent = (+val === 0 ? 'OFF' : val);
    var now = Date.now();
    if (now - lastFlashSend >= 30) { lastFlashSend = now; sendCam('Light', val); }
}

function resetPanTilt() {
    document.getElementById('panSlider').value = 90;
    document.getElementById('tiltSlider').value = 90;
    document.getElementById('panVal').textContent = '90\u00b0';
    document.getElementById('tiltVal').textContent = '90\u00b0';
    sendCam('Pan', 90); sendCam('Tilt', 90);
}

// ===== Car control =====

function pressStart(cmd) {
    send(cmd);
    holdInterval = setInterval(function() { send(cmd); }, 30);
}

function pressEnd() {
    if (holdInterval) { clearInterval(holdInterval); holdInterval = null; }
    send('stop');
}

// ===== Mode =====

function setMode(m) {
    send(m); currentMode = m; updateModeUI(m);
}

function updateModeUI(m) {

```

```

var badge = document.getElementById('modeBadge');
badge.className = 'mode-badge ' + (m==='auto' ? 'badge-auto' : 'badge-manual');
badge.textContent = m==='auto' ? 'AUTO MODE' : 'MANUAL MODE';
document.getElementById('servoManual').style.display = (m==='manual') ? 'block' : 'none';
document.getElementById('servoAuto').style.display = (m==='auto') ? 'block' : 'none';
}

// ===== LED / Buzzer =====
function toggleLed(color, state) {
  if (color === 'red') {
    send(state==='on' ? 'ledrednon' : 'ledredoff');
    document.getElementById('btnRedOn').className = 'action-btn ' + (state==='on' ? 'btn-active-red' : 'btn-off');
    document.getElementById('btnRedOff').className = 'action-btn btn-off';
  } else {
    send(state==='on' ? 'ledgrnon' : 'ledgrnoff');
    document.getElementById('btnGrnOn').className = 'action-btn ' + (state==='on' ? 'btn-active-green' : 'btn-off');
    document.getElementById('btnGrnOff').className = 'action-btn btn-off';
  }
}

function toggleBuzz(state) {
  send('buzzer' + state);
  document.getElementById('btnBuzzOn').className = 'action-btn ' + (state==='on' ? 'btn-active-buzz' : 'btn-off');
  document.getElementById('btnBuzzOff').className = 'action-btn btn-off';
}

// ===== Nozzle Servo =====
function onServoSlide(val) {
  document.getElementById('servoVal').textContent = val + '\u00b0';
}

```

```

var now = Date.now();

if (now - lastServoSend >= 30) { lastServoSend = now; send('servo,' + val); }
}

// ===== Status UI =====

function updateUI(d) {
    setText('temp', d.temp + '\u00b0C', 'val-info');
    setText('hum', d.hum + '%', 'val-info');
    setText('smoke', d.smoke, d.smoke==='CLEAR' ? 'val-safe' : 'val-danger');
    setText('fire', d.fire, d.fire==='SAFE' ? 'val-safe' : 'val-danger');
    setText('pump', d.pump, d.pump==='OFF' ? 'val-neutral' : 'val-warn');
    setText('robot', d.robot, 'val-neutral');

    if (d.mode && d.mode.toLowerCase() !== currentMode) {
        currentMode = d.mode.toLowerCase();
        updateModeUI(currentMode);
    }

    if (d.mode === 'AUTO') {
        document.getElementById('servoAutoVal').textContent = d.servo + '\u00b0';
    }

    if (d.camip && d.camip.length > 0) {
        if (lastCamIp !== d.camip) {
            lastCamIp = d.camip;
            document.getElementById('camContainer').innerHTML =
                "<iframe src='http://' + d.camip + '/stream' style='width:100%;height:280px;border:none;border-radius:8px;background:#000;display:block;'></iframe>"
                + "<div class='cam-status'>CAM IP: " + d.camip + "</div>";
            initCamWS(d.camip);
        }
    }
}

```

```

    } else {
        if (lastCamIp !== "") {
            lastCamIp = "";
            document.getElementById('camContainer').innerHTML =
                "<div class='cam-warning'>&#9888; ESP32-CAM \u09aa\u09be\u0993\u09af\u09bc\u09be\u09af\u09be\u09af\u09bc\u09a8\u09bf\u0964</div>";
        }
    }
}

function setText(id, val, cls) {
    var el = document.getElementById(id);
    el.textContent = val;
    el.className = 'status-value ' + cls;
}

initWS();
updateModeUI('auto');
</script>
</body>
</html>
)HTMLPAGE";

// ===== STATUS PUSH =====
unsigned long lastStatusPush = 0;

void pushStatus() {
    if (millis() - lastStatusPush < 500) return;
    lastStatusPush = millis();
    if (wsCtrl.count() > 0) wsCtrl.textAll(buildStatus());
}

```

```

}

// ===== SETUP =====

void setup() {
  Serial.begin(115200);
  Serial.println("\nFire Robot starting...");

  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);

  pinMode(PUMP, OUTPUT); digitalWrite(PUMP, RELAY_OFF);
  pinMode(RED_LED, OUTPUT); digitalWrite(RED_LED, LOW);
  pinMode(GREEN_LED, OUTPUT); digitalWrite(GREEN_LED, LOW);
  pinMode(BUZZER, OUTPUT); digitalWrite(BUZZER, LOW);

  pinMode(FLAME_LEFT, INPUT);
  pinMode(FLAME_MIDDLE, INPUT);
  pinMode(FLAME_RIGHT, INPUT);
  pinMode(MQ2_PIN, INPUT);

  sprayServo.attach(SERVO_PIN);
  sprayServo.write(90);
  dht.begin();

  WiFi.begin(ssid, password);
  Serial.print("Connecting WiFi");
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println();
  Serial.print("Fire Robot IP: http://"); Serial.println(WiFi.localIP());
}

```

```

if (!MDNS.begin("firerobot")) {
    Serial.println("mDNS FAILED!");
} else {
    Serial.println("mDNS OK — firerobot.local");
    MDNS.addService("http", "tcp", 80);
}

delay(3000);
resolveCamIP();

wsCtrl.onEvent(onCtrlEvent);
server.addHandler(&wsCtrl);

server.on("/", HTTP_GET, [](AsyncWebServerRequest* req){
    req->send_P(200, "text/html", htmlPage);
});
server.onNotFound([](AsyncWebServerRequest* req){
    req->send(404, "text/plain", "Not found");
});

server.begin();
Serial.println("HTTP server started");
}

// ===== LOOP =====

unsigned long lastCamCheck = 0;
unsigned long lastDHTRead = 0;

void loop() {

```

```

wsCtrl.cleanupClients();

if (millis() - lastDHTRead > 2000) {
    lastDHTRead = millis();
    float h = dht.readHumidity();
    float t = dht.readTemperature();
    if (!isnan(h) && !isnan(t)) { humidity = h; temperature = t; }
}

leftValue  = analogRead(FLAME_LEFT);
middleValue = analogRead(FLAME_MIDDLE);
rightValue = analogRead(FLAME_RIGHT);
int smokeVal = analogRead(MQ2_PIN);

bool fireDetected = (leftValue < FIRE_THRESHOLD || middleValue < FIRE_THRESHOLD ||
rightValue < FIRE_THRESHOLD);
bool smokeDetected = (smokeVal > SMOKE_THRESHOLD);

Serial.print("L:"); Serial.print(leftValue);
Serial.print(" M:"); Serial.print(middleValue);
Serial.print(" R:"); Serial.println(rightValue);

if (autoMode) {
    smokeStatus = smokeDetected ? "SMOKE!" : "CLEAR";

    if (fireDetected || smokeDetected) {
        digitalWrite(RED_LED, HIGH); digitalWrite(GREEN_LED, LOW); digitalWrite(BUZZER, HIGH);
    } else {
        digitalWrite(RED_LED, LOW); digitalWrite(GREEN_LED, HIGH); digitalWrite(BUZZER, LOW);
    }
}

```

```

if (fireDetected) {
  fireStatus = "FIRE!";
  firefightMode();
} else {
  fireStatus = "SAFE";
  pumpOff(); stopMoving();
  servoAngle = 90; sprayServo.write(90);
}
}

pushStatus();

if (millis() - lastCamCheck > 30000) {
  lastCamCheck = millis();
  if (CAM_IP.length() == 0) resolveCamIP();
}
}

```

CAMERA CODE : ESP32 CAM

```
/*
 * =====
 * ESP32-CAM — Camera Streaming + Pan/Tilt + mDNS
 * Board: AI-Thinker ESP32-CAM
 *
 * পরিবর্তন:
 * - /stream route যোগ — শুধু camera canvas (iframe এর জন্য)
 * - fb_count: 2, FRAMESIZE_QVGA — reconnect fix
 * - Queue full হলে frame skip, block না করা
 * - PAN Servo → GPIO 13
 * - TILT Servo → GPIO 12
 * - FLASH LED → GPIO 4
 * =====
 */

#include "esp_camera.h"
#include <Arduino.h>
#include <WiFi.h>
#include <ESPmDNS.h>
#include <AsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <ESP32Servo.h>
#include <sstream>

// ===== WIFI =====

const char* ssid = "Net_Error";
```

```

const char* password = "123456780";

#define MDNS_NAME "firecam"

// ===== PAN/TILT =====

#define PAN_PIN 13
#define TILT_PIN 12

Servo panServo;
Servo tiltServo;

// ===== FLASH LED =====

#define LIGHT_PIN 4
#define PWMFreq 1000
#define PWMResolution 8
#define PWMLightChannel 5

// ===== CAMERA PINS (AI-Thinker) =====

#define PWDN_GPIO_NUM 32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27
#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18

```

```

#define Y2_GPIO_NUM    5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM  23
#define PCLK_GPIO_NUM  22

// ===== SERVER =====

AsyncWebServer server(80);
AsyncWebSocket wsCamera("/Camera");
AsyncWebSocket wsInput("/CarInput");

uint32_t cameraClientId = 0;

// ===== STREAM PAGE (🖼️ canvas) =====
const char* streamPage PROGMEM = R"STREAMPAGE(
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { background: #000; display: flex; align-items: center; justify-content: center; width: 100%; height: 100vh; overflow: hidden; }
    canvas { width: 100%; height: 100%; object-fit: contain; display: block; }
  </style>
</head>
<body>
  <canvas id="cam" width="320" height="240"></canvas>
</script>
  var ws = new WebSocket('ws://' + window.location.hostname + '/Camera');

```

```

ws.binaryType = 'arraybuffer';

var canvas = document.getElementById('cam');
var ctx = canvas.getContext('2d');


ws.onmessage = function(e) {
    var blob = new Blob([e.data], {type: 'image/jpeg'});
    var url = URL.createObjectURL(blob);
    var img = new Image();
    img.onload = function() {
        canvas.width = img.naturalWidth || 320;
        canvas.height = img.naturalHeight || 240;
        ctx.drawImage(img, 0, 0);
        URL.revokeObjectURL(url);
    };
    img.onerror = function() { URL.revokeObjectURL(url); };
    img.src = url;
};


ws.onclose = function() { setTimeout(function(){ location.reload(); }, 2000); };
</script>
</body>
</html>
)STREAMPAGE";


// ===== FULL PAGE (original) =====
const char* htmlPage PROGMEM = R"HTMLPAGE(
<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8">

```

```
<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no">
```

```
<style>
```

```
* { box-sizing: border-box; margin: 0; padding: 0; }
```

```
body {
```

```
  background: #0d0d0d;
```

```
  color: #eee;
```

```
  font-family: Arial, sans-serif;
```

```
  padding: 10px;
```

```
  -webkit-user-select: none;
```

```
  user-select: none;
```

```
}
```

```
.label {
```

```
  font-size: 11px;
```

```
  color: #aaa;
```

```
  margin-bottom: 4px;
```

```
  text-transform: uppercase;
```

```
  letter-spacing: 0.5px;
```

```
  display: flex;
```

```
  justify-content: space-between;
```

```
}
```

```
.slider-row { margin-bottom: 12px; }
```

```
input[type=range] {
```

```
  -webkit-appearance: none;
```

```
  width: 100%;
```

```
  height: 12px;
```

```
  border-radius: 6px;
```

```
  background: #2a2a2a;
```

```
  outline: none;
```

```
}
```

```

input[type=range]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 24px;
  height: 24px;
  border-radius: 50%;
  background: #ff6b35;
  cursor: pointer;
}

.val { color: #ff6b35; font-weight: bold; }

.reset-btn {
  width: 100%;
  padding: 10px;
  margin-top: 4px;
  background: #1e3a5f;
  color: #60a5fa;
  border: 1px solid #2563eb;
  border-radius: 8px;
  font-size: 13px;
  cursor: pointer;
}

.reset-btn:active { background: #2563eb; color: #fff; }

.status {
  font-size: 11px;
  color: #555;
  margin-bottom: 10px;
  display: flex;
  align-items: center;
  gap: 6px;
}

.dot { width: 8px; height: 8px; border-radius: 50%; background: #f87171; flex-shrink: 0; }

```

```

        .dot.on { background: #4ade80; }
    </style>
</head>
<body>
    <div class="status">
        <span class="dot" id="dot"></span>
        <span id="statusText">WebSocket connecting...</span>
    </div>

    <canvas id="cameraCanvas" width="320" height="240"
        style="width:100%;height:240px;border-radius:8px;background:#111;margin-
        bottom:12px;display:block;">
    </canvas>

    <div class="slider-row">
        <div class="label">
            <span>⌕ Pan ⌕</span>
            <span class="val" id="panVal">90°</span>
        </div>
        <input type="range" min="0" max="180" value="90" id="Pan"
            oninput="send('Pan',this.value); document.getElementById('panVal').textContent=this.value+'°'">
        </div>

    <div class="slider-row">
        <div class="label">
            <span>⌕ Tilt ⌕</span>
            <span class="val" id="tiltVal">90°</span>
        </div>
        <input type="range" min="0" max="180" value="90" id="Tilt"
            oninput="send('Tilt',this.value); document.getElementById('tiltVal').textContent=this.value+'°'">

```

```
</div>
```

```
<div class="slider-row">
```

```
  <div class="label">
```

```
    <span>Flash Light</span>
```

```
    <span class="val" id="lightVal">OFF</span>
```

```
  </div>
```

```
  <input type="range" min="0" max="255" value="0" id="Light"
```

```
    oninput="send('Light',this.value);
```

```
document.getElementById('lightVal').textContent=(+this.value===0?'OFF':this.value)">
```

```
</div>
```

```
<button class="reset-btn" onclick="resetCenter()">Pan/Tilt Center</button>
```

```
<script>
```

```
var host = window.location.hostname;
```

```
var wsCam = null;
```

```
var wsCtrl = null;
```

```
var ctrlOk = false;
```

```
var canvas = document.getElementById('cameraCanvas');
```

```
var ctx = canvas.getContext('2d');
```

```
function initCam() {
```

```
  wsCam = new WebSocket('ws://' + host + '/Camera');
```

```
  wsCam.binaryType = 'arraybuffer';
```

```
  wsCam.onopen = function() { setStatus(true, 'Camera connected'); };
```

```
  wsCam.onclose = function() {
```

```
    setStatus(false, 'Camera disconnected — reconnecting...');
```

```
    setTimeout(initCam, 2000);
```

```
};
```

```

wsCam.onerror = function() { setStatus(false, 'Camera error'); };
wsCam.onmessage = function(e) {
    var blob = new Blob([e.data], {type: 'image/jpeg'});
    var url = URL.createObjectURL(blob);
    var img = new Image();
    img.onload = function() {
        canvas.width = img.naturalWidth || 320;
        canvas.height = img.naturalHeight || 240;
        ctx.drawImage(img, 0, 0);
        URL.revokeObjectURL(url);
    };
    img.onerror = function() { URL.revokeObjectURL(url); };
    img.src = url;
};
}

function initCtrl() {
    wsCtrl = new WebSocket('ws://' + host + '/CarInput');
    wsCtrl.onopen = function() {
        ctrlOk = true;
        send('Pan', document.getElementById('Pan').value);
        send('Tilt', document.getElementById('Tilt').value);
        send('Light', document.getElementById('Light').value);
    };
    wsCtrl.onclose = function() { ctrlOk = false; setTimeout(initCtrl, 2000); };
    wsCtrl.onerror = function() { ctrlOk = false; };
    wsCtrl.onmessage = function() {};
}

function send(key, val) {

```

```

    if (ctrlOk && wsCtrl && wsCtrl.readyState === WebSocket.OPEN)
        wsCtrl.send(key + ',' + val);
}

function setStatus(ok, msg) {
    document.getElementById('dot').className = 'dot' + (ok ? ' on' : '');
    document.getElementById('statusText').textContent = msg;
}

function resetCenter() {
    document.getElementById('Pan').value = 90;
    document.getElementById('Tilt').value = 90;
    document.getElementById('panVal').textContent = '90°';
    document.getElementById('tiltVal').textContent = '90°';
    send('Pan', 90);
    send('Tilt', 90);
}

initCam();
initCtrl();
</script>
</body>
</html>
)HTMLPAGE";

// ===== WEBSOCKET HANDLERS =====

void onInputEvent(AsyncWebSocket* s, AsyncWebSocketClient* client,
    AwsEventType type, void* arg, uint8_t* data, size_t len)
{
    switch (type) {

```

```

case WS_EVT_CONNECT:
    Serial.printf("Ctrl WS connected #%u\n", client->id());
    break;
case WS_EVT_DISCONNECT:
    Serial.printf("Ctrl WS disconnected #%u\n", client->id());
    panServo.write(90);
    tiltServo.write(90);
    ledcWrite(PWMLightChannel, 0);
    break;
case WS_EVT_DATA: {
    AwsFrameInfo* info = (AwsFrameInfo*)arg;
    if (info->final && info->index == 0 &&
        info->len == len && info->opcode == WS_TEXT)
    {
        std::string msg((char*)data, len);
        std::istringstream ss(msg);
        std::string key, val;
        std::getline(ss, key, ',');
        std::getline(ss, val, ',');
        int v = atoi(val.c_str());
        Serial.printf(" → %s = %d\n", key.c_str(), v);
        if (key == "Pan") panServo.write(v);
        else if (key == "Tilt") tiltServo.write(v);
        else if (key == "Light") ledcWrite(PWMLightChannel, v);
    }
    break;
}
default: break;
}
}

```

```

void onCameraEvent(AsyncWebSocket* s, AsyncWebSocketClient* client,
                  AwsEventType type, void* arg, uint8_t* data, size_t len)
{
    switch (type) {
        case WS_EVT_CONNECT:
            cameraClientId = client->id();
            Serial.printf("Camera WS connected #%u\n", client->id());
            break;
        case WS_EVT_DISCONNECT:
            cameraClientId = 0;
            Serial.printf("Camera WS disconnected #%u\n", client->id());
            break;
        default: break;
    }
}

```

// ===== CAMERA SETUP =====

```

void setupCamera() {
    camera_config_t cfg;
    cfg.ledc_channel = LEDC_CHANNEL_4;
    cfg.ledc_timer   = LEDC_TIMER_2;
    cfg.pin_d0       = Y2_GPIO_NUM;
    cfg.pin_d1       = Y3_GPIO_NUM;
    cfg.pin_d2       = Y4_GPIO_NUM;
    cfg.pin_d3       = Y5_GPIO_NUM;
    cfg.pin_d4       = Y6_GPIO_NUM;
    cfg.pin_d5       = Y7_GPIO_NUM;
    cfg.pin_d6       = Y8_GPIO_NUM;
    cfg.pin_d7       = Y9_GPIO_NUM;
}

```

```

cfg.pin_xclk    = XCLK_GPIO_NUM;
cfg.pin_pclk    = PCLK_GPIO_NUM;
cfg.pin_vsync    = VSYNC_GPIO_NUM;
cfg.pin_href    = HREF_GPIO_NUM;
cfg.pin_sscb_sda = SIOD_GPIO_NUM;
cfg.pin_sscb_scl = SIOC_GPIO_NUM;
cfg.pin_pwdn    = PWDN_GPIO_NUM;
cfg.pin_reset    = RESET_GPIO_NUM;
cfg.xclk_freq_hz = 20000000;
cfg.pixel_format = PIXFORMAT_JPEG;
cfg.frame_size   = FRAMESIZE_QVGA; // ☒ VGA → QVGA (reconnect fix)
cfg.jpeg_quality = 12;
cfg.fb_count     = 2;           // ☒ 1 → 2 (reconnect fix)


esp_err_t err = esp_camera_init(&cfg);
if (err != ESP_OK) {
    Serial.printf("Camera init FAILED: 0x%x\n", err);
    return;
}
if (psramFound()) {
    heap_caps_malloc_extmem_enable(20000);
    Serial.println("PSRAM enabled");
}
Serial.println("Camera OK");
}


// ===== SEND FRAME =====
void sendFrame() {
    if (cameraClientId == 0) return;

```

```

// Queue full হলে frame skip করো, block না করে
AsyncWebSocketClient* c = wsCamera.client(cameraClientId);
if (!c || c->queueIsFull()) return; // ☒ reconnect fix

camera_fb_t* fb = esp_camera_fb_get();
if (!fb) return;

wsCamera.binary(cameraClientId, fb->buf, fb->len);
esp_camera_fb_return(fb);
}

// ===== SETUP =====
void setup() {
  Serial.begin(115200);
  Serial.println("\nESP32-CAM starting...");

  ledcSetup(PWMLightChannel, PWMFreq, PWMResolution);
  ledcAttachPin(LIGHT_PIN, PWMLightChannel);
  ledcWrite(PWMLightChannel, 0);
  Serial.println("Light setup done");

  WiFi.begin(ssid, password);
  Serial.print("Connecting WiFi");
  int tries = 0;
  while (WiFi.status() != WL_CONNECTED && tries < 30) {
    delay(500); Serial.print("."); tries++;
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\n=====");
    Serial.print(" ESP32-CAM IP: http://");
  }
}

```

```

Serial.println(WiFi.localIP());

Serial.println("=====");
} else {
    Serial.println("\nWiFi FAILED");
    return;
}

if (!MDNS.begin(MDNS_NAME)) {
    Serial.println("mDNS FAILED!");
} else {
    Serial.println("mDNS OK! → http://firecam.local");
    MDNS.addService("http", "tcp", 80);
}

// Routes
server.on("/", HTTP_GET, [](AsyncWebServerRequest* req){
    req->send_P(200, "text/html", htmlPage);
});

// ☒ /stream —  camera canvas (main dashboard এর iframe এর জন্য)
server.on("/stream", HTTP_GET, [](AsyncWebServerRequest* req){
    req->send_P(200, "text/html", streamPage);
});

server.onNotFound([](AsyncWebServerRequest* req){
    req->send(404, "text/plain", "Not found");
});

wsCamera.onEvent(onCameraEvent);
wsInput.onEvent(onInputEvent);
server.addHandler(&wsCamera);
server.addHandler(&wsInput);

```

```

server.begin();

Serial.println("HTTP server started");


setupCamera();


// ☒ Camera init এর পরে servo attach
panServo.attach(PAN_PIN);
tiltServo.attach(TILT_PIN);
panServo.write(90);
tiltServo.write(90);
Serial.println("Servos attached");
}


// ===== LOOP =====
void loop() {
  wsCamera.cleanupClients();
  wsInput.cleanupClients();
  sendFrame();
}

```